

DEVOPS PRACTICAL REFERENCE SERIES

PART 04

CI/CD Fundamentals & GitHub Actions

From pipeline theory to production-grade workflows -- OIDC cloud auth, reusable workflows, supply-chain security, and real deployment pipelines.

CI/CD

GitHub Actions

OIDC

Pipeline Security

DORA Metrics

Troubleshooting

Audience: DevOps engineers with ~2 years of experience

Format: Deep-dive practical reference — concepts, architecture, configuration, commands, troubleshooting, security, automation, and hands-on labs.

How to use: Read sequentially. Each part is self-contained but builds on earlier parts in the series.

Table of Contents

1. Introduction	5
1.1 What This Part Covers	5
1.2 Why CI/CD Exists	5
2. CI/CD Fundamentals	6
2.1 Continuous Integration	6
2.2 Continuous Delivery vs Continuous Deployment	6
2.3 The Canonical Pipeline Stages	7
2.4 Artifacts and Immutability	8
3. GitHub Actions Architecture	10
3.1 What It Is and Why It Exists	10
3.2 Core Concepts and How They Relate	10
3.3 What Actually Happens Behind the Scenes	11
4. Writing Workflows	12
4.1 Anatomy of a Workflow File	12
4.2 Trigger Events (<code>on:</code>)	13
4.3 Contexts and Expressions	14
4.4 Job Dependencies and Parallelism	15
4.5 Matrix Builds	15
4.6 Concurrency Control	16
5. Runners	17
5.1 GitHub-Hosted Runners	17
5.2 Self-Hosted Runners	17
5.3 Runner Groups and Scaling Self-Hosted Runners	18
6. Actions Marketplace and Reusable Workflows	19
6.1 Using Marketplace Actions	19
6.2 Composite Actions	19
6.3 Reusable Workflows	20
6.4 Writing a Custom JavaScript/Docker Action	21
7. Secrets, Environments, and OIDC	23
7.1 Repository and Organization Secrets	23
7.2 Environments and Protection Rules	23
7.3 The Built-In GITHUB_TOKEN	24
7.4 OIDC: Eliminating Long-Lived Cloud Credentials	25
8. Artifacts and Caching	27
8.1 Artifacts: Passing Data Between Jobs	27
8.2 Dependency Caching	27

9. Build/Test/Deploy Pipeline Patterns	29
9.1 A Complete, Realistic CI Pipeline	30
9.2 Adding Deployment Stages	32
9.3 Monorepo-Aware Pipelines (Path Filtering)	33
9.4 Tagging Artifacts with Git SHA for Traceability	33
10. Advanced Workflow Patterns	35
10.1 Job Outputs	35
10.2 Dynamic Matrices	35
10.3 Conditional Logic Patterns	36
10.4 Composing It All: A Release Workflow	36
11. CI/CD Security	37
11.1 The CI/CD Pipeline as an Attack Surface	37
11.2 Least Privilege, Applied to Actions	37
11.3 The pull_request_target Trap, Explained in Full	37
11.4 Supply-Chain Basics for Pipelines	38
11.5 Branch Protection as a CI/CD Security Control	38
12. Monitoring and Observability of Pipelines	39
12.1 Built-In Visibility	39
12.2 Notifications	39
12.3 Pipeline Metrics Worth Tracking	40
13. Troubleshooting GitHub Actions	41
13.1 A Systematic Approach	41
13.2 Common Failure Scenarios	41
Scenario: “Works locally, fails in CI”	41
Scenario: A step can’t find a file that “should” be there	41
Scenario: A secret isn’t available in a step	41
Scenario: A job that should run in parallel seems to wait	42
Scenario: Matrix build partially fails and you can’t tell which combination	42
13.3 Debug Logging	42
14. Best Practices and Anti-Patterns	43
15. Hands-On Labs	44
Lab 1 — Your First Multi-Job Workflow	44
Lab 2 — Matrix Builds and Path Filtering	44
Lab 3 — Reusable Workflow	44
Lab 4 — Secrets, Environments, and Approval Gates	44
Lab 5 — OIDC to AWS (or conceptual walkthrough if no AWS account)	45
Lab 6 — Troubleshooting Drill	45
16. Command & Reference Cheat Sheet	46
16.1 gh CLI for Actions	46
16.2 Common Trigger Patterns	46

16.3 Key Contexts	46
16.4 Security Quick Reference	47
17. Production Readiness Checklist	48

1. Introduction

1.1 What This Part Covers

Part 04 covers CI/CD as a discipline — the practices, pipeline stages, and failure modes that apply regardless of which tool implements them — and GitHub Actions specifically, as the most widely used modern CI/CD platform. Jenkins, covered separately in Part 05, implements the same underlying CI/CD concepts with a very different architecture (self-hosted, plugin-driven, older), so understanding the concepts here first makes Jenkins far easier to learn as “the same ideas, different implementation” rather than starting from scratch.

HOW TO USE THIS PART

If you already write GitHub Actions workflows regularly, skim Sections 2-4 and focus on Section 7 (secrets/OIDC), Section 11 (CI/CD security), and Section 13 (troubleshooting) — these are where most 2-3 year engineers have real gaps, particularly around why long-lived cloud credentials in CI are a serious anti-pattern and what OIDC actually replaces them with.

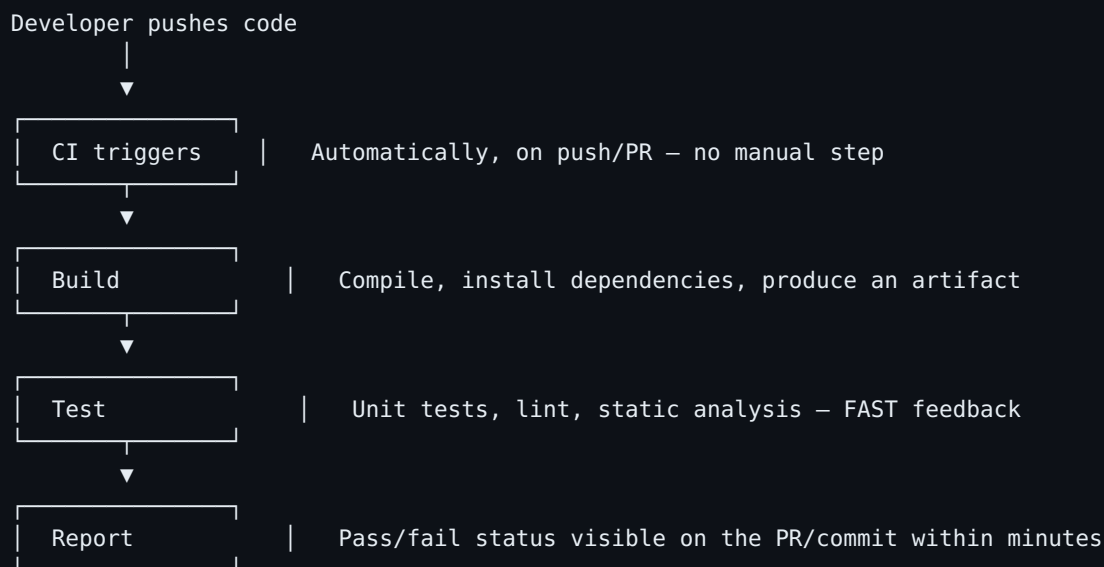
1.2 Why CI/CD Exists

Before CI/CD, integrating code from multiple engineers was a periodic, painful event — “integration hell” — where weeks of divergent work collided at once. **Continuous Integration (CI)** solves this by integrating (merging and testing) every change immediately and automatically, catching conflicts and regressions within minutes of being introduced, not weeks later. **Continuous Delivery/Deployment (CD)** extends this automation through to actually shipping validated changes to production, removing manual, error-prone, inconsistent release processes. Together, CI/CD is what makes small, frequent, low-risk releases possible — the foundation the trunk-based development and GitHub Flow strategies from Part 02 actually depend on to be safe.

2. CI/CD Fundamentals

2.1 Continuous Integration

CI means every code change is automatically built and tested as soon as it's pushed — the core promise is that `main` (or any actively developed branch) stays in a known-good, always-buildable state, and regressions are caught within minutes of being introduced, while the context is still fresh in the author's mind.



2.2 Continuous Delivery vs Continuous Deployment

These two terms are frequently conflated, but the distinction is a real, important architectural decision:

	Continuous Delivery	Continuous Deployment
What it means	Every change that passes CI is automatically prepared/ packaged for release and is ALWAYS in a deployable state — but a human explicitly approves/triggers the actual production release	Every change that passes CI is automatically deployed to production with no human approval step
Human involvement	A deliberate “go” decision (a button click, an approval gate) before prod	None — a merged, passing change ships automatically
Typical fit	Regulated environments, customer-facing products needing release timing control, teams still building confidence in their test coverage	High-velocity teams with strong automated test coverage, feature flags, and fast rollback capability

WHY THIS DISTINCTION IS A COMMON INTERVIEW QUESTION

Both are legitimate, mature practices — the “better” one depends entirely on the organization’s risk tolerance, regulatory requirements, and test automation maturity. A team with thin test coverage practicing continuous DEPLOYMENT is actually taking on significant risk, not demonstrating sophistication; a team with excellent test coverage and feature-flag discipline practicing continuous DELIVERY (requiring manual approval) is leaving velocity on the table unnecessarily. Be ready to articulate this tradeoff rather than asserting one is simply more “advanced.”

2.3 The Canonical Pipeline Stages

```
Source → Build → Test → Package → Deploy (staging) → Test (integration/e2e)
→ Deploy (production)
```

Stage	Purpose	Typical duration target
Source	Trigger on push/PR/tag	Instant
Build	Compile code, install dependencies, produce a deployable artifact (binary, container image, package)	Seconds to a few minutes
Test	Unit tests, linting, static analysis, security scanning (SAST)	Minutes — this is the stage developers wait on directly, keep it FAST
Package	Produce the final versioned, immutable artifact (e.g., a tagged container image pushed to a registry)	Seconds to minutes
Deploy to staging	Automatically deploy to a non-production environment	Minutes
Integration/E2E tests	Slower, more realistic tests against a real deployed environment	Minutes to tens of minutes
Deploy to production	Either automatic (Continuous Deployment) or gated by approval (Continuous Delivery)	Minutes

PRODUCTION PRACTICE: THE “FAST FEEDBACK FIRST” PRINCIPLE

Order pipeline stages from fastest/cheapest to slowest/most expensive, and fail FAST — a linter that takes 10 seconds should run before a full integration test suite that takes 20 minutes, so a trivial syntax error is caught immediately rather than after a long wait. This ordering principle alone is one of the highest-leverage things you can do to keep a team’s CI feedback loop fast and genuinely useful rather than something people learn to ignore or work around.

2.4 Artifacts and Immutability

A **build artifact** (a compiled binary, a container image, a packaged library) produced once during CI should be the EXACT same artifact promoted through every subsequent environment (staging, then production) — never rebuilt per environment. This “build once, promote the same artifact” principle is foundational:

ANTI-PATTERN: REBUILDING PER ENVIRONMENT

Running a fresh build for staging and a SEPARATE fresh build for production — even from the identical commit — risks subtle differences (a dependency version that moved between builds, a flaky build step, a different build-time environment variable) silently producing a different artifact than the one that was actually tested in staging. The artifact

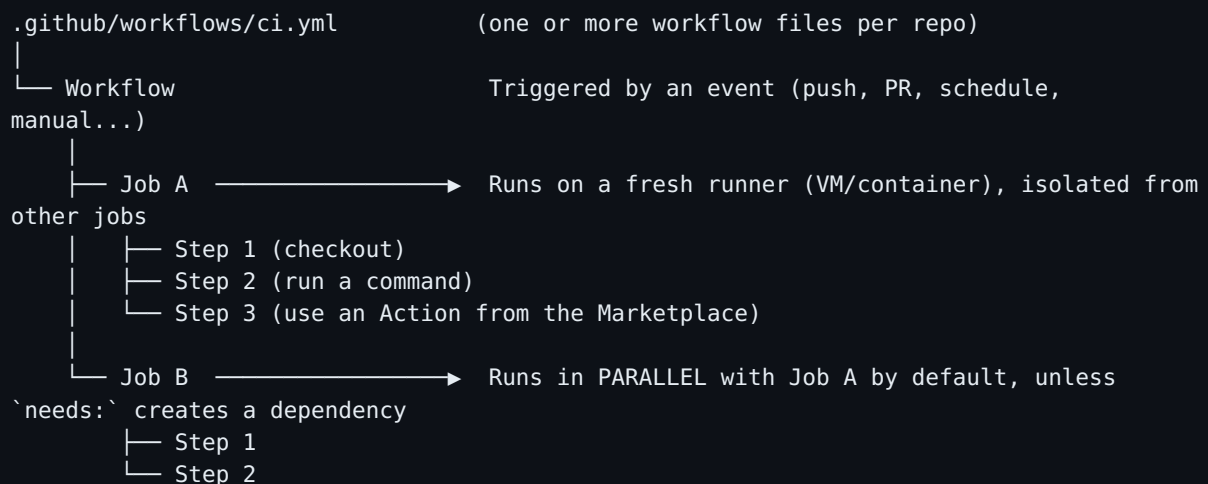
that passed every test should be the EXACT same bytes that reach production — typically achieved by tagging a container image with an immutable identifier (a Git SHA, Section 9.4) and only ever re-tagging/promoting that same image, never rebuilding.

3. GitHub Actions Architecture

3.1 What It Is and Why It Exists

GitHub Actions is GitHub's built-in CI/CD platform: YAML-defined automation that runs in response to repository events (pushes, PRs, releases, schedules, manual triggers), executing on either GitHub-hosted or self-hosted compute. Its tight integration with the rest of the GitHub platform (PRs, Checks API, `GITHUB_TOKEN`, the Actions Marketplace) is its core differentiator versus a standalone CI tool like Jenkins.

3.2 Core Concepts and How They Relate



Concept	What it is
Workflow	A YAML file in <code>.github/workflows/</code> , defining what triggers it and what jobs run
Event	What triggers the workflow: <code>push</code> , <code>pull_request</code> , <code>schedule</code> , <code>workflow_dispatch</code> (manual), <code>release</code> , and many more
Job	A set of steps that run together on the SAME runner instance; jobs run in parallel by default
Step	A single task within a job: either a shell command (<code>run:</code>) or a reusable Action (<code>uses:</code>)
Action	A reusable unit of automation — either from the Marketplace, a Docker container, composite steps, or your own repo
Runner	The actual compute (VM or self-hosted machine) that executes a job's steps

3.3 What Actually Happens Behind the Scenes

1. An event occurs (e.g., a push to `main`).
2. GitHub's Actions service matches the event against every workflow file's `on:` trigger conditions across the repo.
3. For each matching workflow, GitHub provisions a fresh runner VM (GitHub-hosted) or dispatches to an available self-hosted runner for each job.
4. The runner checks out the repo (if a checkout step is included — it is NOT automatic), then executes each step in sequence, in the SAME filesystem/shell environment within that job.
5. Each job's logs, timing, and pass/fail status are reported back and surfaced directly on the commit/PR via the **Checks API** — this is the exact mechanism behind “required status checks” in branch protection (Part 02, Section 14.4).
6. The runner VM is destroyed after the job completes (GitHub-hosted) — nothing persists between runs unless explicitly cached or uploaded as an artifact (Section 8).

WHY EACH JOB GETS A FRESH ENVIRONMENT

Job isolation (a brand-new VM per job, by default) guarantees reproducibility — no leftover state from a previous run can silently affect the next one. This is also exactly why steps WITHIN the same job can share files/state freely (same filesystem), but separate JOBS cannot, without explicitly passing data via artifacts (Section 8.1) or job outputs (Section 10.1).

4. Writing Workflows

4.1 Anatomy of a Workflow File

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:          # allows manually triggering from the Actions tab/API
  inputs:
    debug:
      description: "Enable debug logging"
      type: boolean
      default: false

env:
  NODE_VERSION: "20"          # available to every job/step in this workflow

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4          # NOT automatic – every workflow needing the
repo's files must include this

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${ env.NODE_VERSION }
          cache: "npm"                    # built-in dependency caching (Section 8.2)

      - name: Install dependencies
        run: npm ci                      # NOT `npm install` – ci uses the lockfile
exactly, faster and reproducible

      - name: Lint
        run: npm run lint

      - name: Run tests
        run: npm test -- --coverage

      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

4.2 Trigger Events (`on:`)

Trigger	Fires when
<code>push</code>	Commits pushed to matching branches/tags
<code>pull_request</code>	A PR is opened, synchronized (new commits pushed), reopened, etc.
<code>pull_request_target</code>	Like <code>pull_request</code> , but runs in the context of the BASE branch, with access to secrets — needed for workflows that must act on forked-repo PRs, but requires extreme care (Section 11.3)
<code>schedule</code>	Cron-syntax schedule (Part 01, Section 12.2 syntax applies) — e.g., nightly builds
<code>workflow_dispatch</code>	Manual trigger from the UI, CLI (<code>gh workflow run</code>), or API — supports typed inputs
<code>release</code>	A GitHub Release is published/created
<code>workflow_call</code>	Makes this workflow reusable/callable BY other workflows (Section 6.2)
<code>repository_dispatch</code>	Custom external trigger via the API — for integrating with systems outside GitHub

```
on:
  push:
    branches: [main, "release/**"]      # glob patterns supported
    paths: ["src/**", "package.json"]   # only trigger if matching files actually
    changed – saves wasted CI runs
    paths-ignore: ["docs/**", "*.md"]
  schedule:
    - cron: "0 2 * * *"                # daily at 02:00 UTC – GitHub Actions
schedules always run in UTC
```

PULL_REQUEST VS PULL_REQUEST_TARGET: A REAL SECURITY DISTINCTION

`pull_request` checks out and runs against the PR's code, but with a RESTRICTED, read-only `GITHUB_TOKEN` and NO access to repository secrets when the PR comes from a fork — this is a deliberate security boundary preventing a malicious fork PR from exfiltrating your secrets. `pull_request_target` runs with the BASE repo's full permissions and secret access, EVEN for fork PRs — which is necessary for some legitimate automation (e.g., auto-labeling) but is also a well-known attack vector if the workflow then checks out and EXECUTES the fork's untrusted code with that elevated access. Never combine `pull_request_target` with checking out and running arbitrary code from the PR branch itself — Section 11.3 covers this exact vulnerability pattern.

4.3 Contexts and Expressions

```
steps:
  - name: Show context info
    run: |
      echo "Event: ${github.event_name}"
      echo "Branch: ${github.ref_name}"
      echo "Actor: ${github.actor}"
      echo "SHA: ${github.sha}"

  - name: Conditional step
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    run: ./deploy.sh

  - name: Use a secret
    run: ./deploy.sh
    env:
      API_KEY: ${secrets.DEPLOY_API_KEY}
```

Context	Provides
<code>github.*</code>	Event payload, repo, ref, SHA, actor, workflow metadata
<code>secrets.*</code>	Configured repository/environment secrets (Section 7.1)
<code>env.*</code>	Environment variables defined at workflow/job/step level
<code>vars.*</code>	Configured (non-secret) repository/organization variables
<code>needs.*</code>	Outputs from jobs this job depends on (Section 10.1)
<code>matrix.*</code>	The current combination's values in a matrix build (Section 4.5)
<code>steps.*</code>	Outputs from previous steps in the SAME job

4.4 Job Dependencies and Parallelism

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: [...]

  test:
    runs-on: ubuntu-latest
    steps: [...]

  build:
    needs: [lint, test]           # waits for BOTH to succeed before starting
    runs-on: ubuntu-latest
    steps: [...]

  deploy:
    needs: build
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps: [...]
```

Without `needs:`, every job in a workflow runs in parallel, on separate runners, by default — this is a deliberate design maximizing speed, but means you must **EXPLICITLY** declare ordering wherever one job's output depends on another's completion.

4.5 Matrix Builds

```
jobs:
  test:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node-version: [18, 20, 22]
      fail-fast:
        false                # by default, ANY matrix failure cancels all others – false lets
                             # every combination finish and report independently
      max-parallel:
        4                    # cap concurrent matrix jobs (useful for limited self-hosted runner
                             # capacity)
    runs-on: ${matrix.os}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${matrix.node-version}
      - run: npm ci && npm test
```

This single job definition expands into **9 separate parallel jobs** (3 OSes × 3 Node versions) — the standard pattern for testing compatibility across multiple runtime versions or platforms without duplicating workflow YAML.

4.6 Concurrency Control

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true      # a NEW push to the same branch automatically
                                # cancels the PREVIOUS still-running workflow for that branch
```

WHY THIS MATTERS AT SCALE

Without concurrency control, pushing 5 commits to a PR in quick succession queues/runs 5 full CI pipelines, wasting runner minutes (real cost on GitHub-hosted runners) on results that are about to be superseded anyway. `cancel-in-progress: true` scoped by branch/ref ensures only the LATEST push's pipeline actually completes, which is almost always the desired behavior for PR validation workflows — but should generally be OFF for deployment workflows, where you don't want an in-progress production deploy cancelled mid-flight by a new push.

5. Runners

5.1 GitHub-Hosted Runners

```
jobs:
  build:
    runs-on: ubuntu-
latest      # also available: windows-latest, macos-latest, and specific versions
like ubuntu-22.04
```

GitHub-hosted runners are ephemeral VMs, freshly provisioned for each job and destroyed immediately after — pre-installed with a large set of common tools (Docker, major language runtimes, cloud CLIs) via the [actions/runner-images](#) project. They're billed per minute (free tier included for public repos and a monthly allowance for private repos), with different per-minute rates for Linux, Windows, and macOS runners (macOS is notably more expensive).

5.2 Self-Hosted Runners

```
jobs:
  build:
    runs-on: [self-hosted, linux, gpu]      # match by custom labels you assign when
registering the runner
```

Self-hosted runners are machines YOU provision and register — VMs, bare metal, or containers you control — that poll GitHub for work and execute jobs.

	GitHub-hosted	Self-hosted
Setup effort	None	You provision, register, patch, and maintain the machine(s)
Cost model	Per-minute billing	Your own infrastructure cost (often cheaper at high, sustained volume)
Customization	Limited to pre-installed tools + setup steps	Full control — custom hardware (GPUs), pre-cached large dependencies, specific OS/tooling
State between runs	None — always a clean VM	Persists by default unless you explicitly clean up — a security and reproducibility consideration
Network access	Public internet only	Can be placed inside a private VPC, reaching internal-only resources directly

CRITICAL SECURITY CONSIDERATION: SELF-HOSTED RUNNERS ON PUBLIC REPOS

Self-hosted runners should almost NEVER be used on public repositories without extremely careful controls — anyone can open a PR, and if your workflow runs on PR events against a self-hosted runner, an attacker can get arbitrary code execution on YOUR infrastructure simply by opening a malicious PR. GitHub’s own documentation strongly warns against this exact configuration. For private/internal repos this risk is much lower (limited to trusted contributors) but still warrants scoping runner permissions tightly and treating the runner machine as part of your security perimeter, not a disposable sandbox, unless you’ve deliberately built it to be ephemeral and isolated per job (e.g., via an autoscaling runner controller that provisions a fresh container per job).

5.3 Runner Groups and Scaling Self-Hosted Runners

For organizations running significant self-hosted capacity, **runner groups** control which repositories can use which runners (access control), and tools like **Actions Runner Controller (ARC)** for Kubernetes auto-scale self-hosted runners as Kubernetes pods, spinning up fresh, isolated runner pods per job and tearing them down immediately after — combining the elasticity/cost model of GitHub-hosted runners with the customization and private-network access of self-hosted ones. This pattern is covered more concretely once Kubernetes fundamentals are established (Part 08 onward).

6. Actions Marketplace and Reusable Workflows

6.1 Using Marketplace Actions

```
steps:
  - uses: actions/checkout@v4                # official GitHub action
  - uses: docker/build-push-action@v5        # popular third-party action,
    by an official Docker org
  - uses: aws-actions/configure-aws-credentials@v4    # AWS's own official action
    (relevant again in Part 20)
```

Actions are referenced as `owner/repo@ref` — the `ref` can be a tag (`@v4`), a branch (`@main`), or a specific commit SHA.

SECURITY PRACTICE: PIN ACTIONS TO A COMMIT SHA, NOT A MUTABLE TAG

A tag like `@v4` can be MOVED by the action's maintainer (or an attacker who compromises their account) to point at different, potentially malicious code without you ever changing your workflow file — this is a real, documented supply-chain attack vector (and the subject of several real incidents in the Actions ecosystem). Pinning to a full commit SHA (`uses: actions/checkout@8f4b7f84...`) guarantees the EXACT code that runs, immune to this class of attack. Many security-conscious organizations enforce this via policy or automated tooling (like Dependabot, which can still propose SHA-to-SHA update PRs while keeping you pinned). This matters enough that Part 26 (CI/CD and supply-chain security) revisits it in full depth.

6.2 Composite Actions

A composite action bundles multiple steps into one reusable, shareable unit — useful for a sequence you repeat across many workflows WITHIN the same repo, or shared across repos if published separately.

```
# .github/actions/setup-project/action.yml
name: "Setup Project"
description: "Checkout, install dependencies, and set up the Node environment"
inputs:
  node-version:
    description: "Node.js version"
    default: "20"
runs:
  using: "composite"
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: "${{ inputs.node-version }}"
        cache: "npm"
    - run: npm ci
      shell: bash
      # REQUIRED for every `run:` step inside a composite
      action
```

```
# Used from another workflow:
steps:
  - uses: actions/checkout@v4
  - uses: ./.github/actions/setup-project
    with:
      node-version: "22"
```

6.3 Reusable Workflows

A reusable workflow is an entire CALLABLE workflow (not just a few steps) — the standard pattern for sharing a full build/test/deploy pipeline across many repositories in an organization, with each caller supplying its own inputs/secrets.

```
# .github/workflows/reusable-deploy.yml
name: Reusable Deploy

on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
      image-tag:
        required: true
        type: string
    secrets:
      DEPLOY_TOKEN:
        required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${ inputs.environment }
    steps:
      - name: Deploy
        run: ./deploy.sh "${ inputs.image-tag }"
        env:
          TOKEN: ${ secrets.DEPLOY_TOKEN }
```

```
# Called from another repo's workflow:
jobs:
  call-deploy:
    uses: my-org/shared-workflows/.github/workflows/reusable-deploy.yml@main
    with:
      environment: production
      image-tag: ${ github.sha }
    secrets:
      DEPLOY_TOKEN: ${ secrets.PROD_DEPLOY_TOKEN }
```

PRODUCTION PRACTICE: CENTRALIZE PIPELINE LOGIC

At any organization with more than a handful of services, maintaining near-identical copy-pasted CI/CD YAML across every repository becomes an enormous maintenance burden — a change to the standard build process means editing dozens of files. Centralizing the actual pipeline LOGIC into reusable workflows in one shared repo, with individual service repos just calling them with their own parameters, is the standard mature pattern: one place to fix a bug or add a security scan, instantly applied everywhere that calls it.

6.4 Writing a Custom JavaScript/Docker Action

For logic too complex for a composite action (needing real program logic, not just shell commands), Actions supports two more action types:

Type	How it works	Best for
Composite	A sequence of existing steps/actions, YAML-only	Repeated step sequences, no real “new” logic
JavaScript action	A Node.js script using the <code>@actions/toolkit</code> packages, runs directly on the runner	Fast-starting custom logic, no container overhead
Docker container action	Packaged as a container image, runs in that container	Logic needing a specific, isolated runtime/toolchain unavailable on standard runners

Most teams never need to write custom JS/Docker actions — composite actions and reusable workflows cover the vast majority of real organizational needs, and the Marketplace covers most individual tool integrations.

7. Secrets, Environments, and OIDC

7.1 Repository and Organization Secrets

```
steps:
  - name: Deploy
    env:
      API_KEY: ${ secrets.API_KEY }
    run: ./deploy.sh
```

Secrets are encrypted values configured in repo/org Settings → Secrets and variables, never visible in logs (GitHub automatically redacts any secret VALUE that appears in output, replacing it with `***`), and accessible only via the `secrets.*` context — never readable back out through the UI once set, only overwritten.

Scope	Where configured	Use case
Repository secret	One specific repo	Secrets unique to one project
Environment secret	Tied to a named environment (Section 7.2) within a repo	Different values per environment (e.g., different DB credentials for staging vs prod)
Organization secret	Shared across many/all repos in the org	Credentials genuinely common across projects (with repo access restrictable)

SECRET REDACTION ISN'T FOOLPROOF

GitHub's automatic log redaction only masks the EXACT string value of a known secret — if your workflow transforms the secret first (base64-encodes it, splits it into parts, logs it character by character in a loop), the redaction can be bypassed entirely, accidentally leaking it in plaintext logs. Never deliberately echo/print a secret for “debugging,” even temporarily — treat any such accident as an immediate credential-rotation event (same principle as Part 02, Section 15.3).

7.2 Environments and Protection Rules

```
jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://app.example.com # shown as a clickable link on the
    deployment in GitHub's UI
    steps:
      - run: ./deploy.sh
```

Configured under repo Settings → Environments, an **environment** (e.g., `staging`, `production`) can require:

- **Required reviewers:** a human must explicitly approve before a job targeting this environment runs — the core mechanism for implementing Continuous DELIVERY (Section 2.2) rather than full auto-deployment.
- **Wait timer:** a mandatory delay before the job proceeds (a “cool-down” / last-chance-to-cancel window).
- **Deployment branch restrictions:** only specific branches (e.g., `main`, or tags matching `v*`) may deploy to this environment at all — prevents an accidental deploy from a random feature branch.

This is the primary mechanism by which GitHub Actions implements approval gates, and it ties environment-scoped secrets directly to that same gated access — a `production` environment secret is literally inaccessible to any job run that hasn’t passed the environment’s protection rules.

7.3 The Built-In GITHUB_TOKEN

```
permissions:
  contents: read          # explicit, minimal permissions – the production-correct
  pull-requests: write    # only if this workflow actually needs to comment on/
  label PRs

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # GITHUB_TOKEN is used automatically/
        implicitly here
      - name: Comment on PR
        uses: actions/github-script@v7
        with:
          github-token: ${ secrets.GITHUB_TOKEN } # also available explicitly
        when a script needs it directly
        script: |
          github.rest.issues.createComment({...})
```

Every workflow run gets an automatically generated, short-lived `GITHUB_TOKEN`, scoped to that one repository and automatically revoked when the job finishes — no manual secret management needed for standard GitHub API interactions (commenting on PRs, creating releases, pushing to the triggering repo). Its DEFAULT permission scope (configurable at the org/repo level, and overridable per-workflow via the `permissions:` key) should be set to the minimum actually required — `contents: read` plus only the specific additional write scopes a given workflow genuinely needs, following least-privilege.

7.4 OIDC: Eliminating Long-Lived Cloud Credentials

Historically, deploying to AWS/Azure/GCP from CI meant storing long-lived cloud credentials (an AWS access key/secret, for example) as a GitHub secret — a high-value, permanent target if ever leaked, with no automatic expiry. **OIDC (OpenID Connect)** federation eliminates this entirely: GitHub Actions can request a short-lived, auto-expiring cloud credential directly from AWS/Azure/GCP at run time, with NO long-lived secret stored in GitHub at all.

```
GitHub Actions job starts
  |
  ▼
GitHub's OIDC provider issues a short-lived, SIGNED JWT token
  identifying: this exact repo, branch/ref, workflow – "this IS really
  a run of org/repo on branch main"
  |
  ▼
Job presents this JWT to AWS STS (sts:AssumeRoleWithWebIdentity)
  |
  ▼
AWS verifies the JWT's signature against GitHub's published OIDC
  keys, and checks it against a TRUST POLICY on an IAM role that says
  "trust tokens from org/repo on branch main, nothing else"
  |
  ▼
AWS issues short-lived, auto-expiring temporary AWS credentials
  (typically ~1 hour) – scoped to exactly the IAM role's permissions
```

```
permissions:
  id-token: write          # REQUIRED – grants this job permission to request an
  OIDC token at all
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-deploy-role
          aws-region: us-east-1
          # NOTE: no access key / secret key anywhere – the role is assumed via the
          # OIDC token exchange above
      - run: aws s3 sync ./dist s3://my-app-bucket/
```

```
// The IAM role's trust policy (AWS side) – restricts WHICH repo/branch can assume
this role
{
  "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/
token.actions.githubusercontent.com" },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": { "token.actions.githubusercontent.com:sub": "repo:my-org/my-
repo:ref:refs/heads/main" }
  }
}
```

WHY THIS IS NOW THE PRODUCTION STANDARD

OIDC federation means there is NO long-lived cloud credential to leak, rotate, or accidentally commit in the first place — every credential is freshly issued per job run and expires automatically within the hour. Combined with a tightly scoped trust policy condition (restricting exactly which repo AND which branch/ref may assume the role), this closes off an entire historically common class of CI/CD breaches (a leaked long-lived AWS key in a CI secret store or, worse, accidentally logged). Every major cloud provider (AWS, Azure, GCP) supports this exact pattern with GitHub Actions, and it should be the DEFAULT choice for any new pipeline touching cloud infrastructure — this exact mechanism is revisited with full AWS IAM specifics in Part 20 and in the dedicated CI/CD security part (26).

8. Artifacts and Caching

8.1 Artifacts: Passing Data Between Jobs

Because each job runs on its own fresh runner (Section 3.3), sharing files between jobs requires explicitly uploading and downloading them as artifacts.

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: ./build.sh # produces ./dist/
      - uses: actions/upload-artifact@v4
        with:
          name: build-output
          path: dist/
          retention-days: 7 # auto-delete after N days (default org/
repo setting otherwise applies)

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: build-output
          path: dist/
      - run: ./deploy.sh dist/
```

Artifacts are also how test reports, coverage results, and build logs get surfaced in the GitHub UI for later download/inspection — distinct from caching (Section 8.2), which exists purely for SPEED, not for passing data between dependent jobs.

8.2 Dependency Caching

```
steps:
  - uses: actions/setup-node@v4
    with:
      node-version: "20"
      cache: "npm" # the simple, built-in way for common package
managers – handles the cache key automatically

# Or, manually, for full control:
  - uses: actions/cache@v4
    with:
      path: ~/.cache/pip
      key: pip-${ runner.os }-${ hashFiles('requirements.txt') }
      restore-keys: |
        pip-${ runner.os }-
```

Caching stores and restores files (typically downloaded dependencies) BETWEEN separate workflow RUNS (not between jobs in the same run — that's what artifacts are for) to avoid re-downloading the same dependencies every single time, often cutting build time dramatically on dependency-heavy projects.

CACHE KEY DESIGN

The `key` should change whenever the cached content SHOULD be invalidated — hashing the lockfile (`hashFiles('package-lock.json')`) is the standard pattern: if dependencies haven't changed, the hash is identical and the cache hits; if they have, the hash changes and a fresh cache is built (and saved under the new key for next time). `restore-keys` provides a fallback to a close-but-not-exact-match cache (e.g., from a previous dependency set) rather than starting completely from scratch, trading a little staleness for a faster partial cache hit.

CACHING IS FOR SPEED, NEVER FOR SECRETS OR SECURITY-SENSITIVE STATE

Caches are NOT strongly isolated from other workflow runs in the same repo in the way secrets are, and have been a vector in real supply-chain attack research (cache poisoning). Never cache anything containing credentials or sensitive data, and treat cache contents as something a sufficiently privileged co-contributor could potentially influence — this is a genuinely more advanced/edge-case concern than the OIDC and SHA-pinning practices above, but worth being aware of as pipelines grow more complex (Part 26 revisits this).

9. Build/Test/Deploy Pipeline Patterns

9.1 A Complete, Realistic CI Pipeline

```

name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20", cache: "npm" }
      - run: npm ci
      - run: npm run lint

  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20", cache: "npm" }
      - run: npm ci
      - run: npm test -- --coverage
      - uses: actions/upload-artifact@v4
        with: { name: coverage, path: coverage/ }

  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run gitleaks
        uses: gitleaks/gitleaks-action@v2 # secret scanning in CI (Part 02,
Section 15.3's local tool, enforced server-side here)
      - name: Dependency vulnerability scan
        run: npm audit --audit-level=high

  build:
    needs: [lint, unit-tests, security-scan]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: |
            myorg/myapp:${{ github.sha }}
            myorg/myapp:latest

```

```
cache-from: type=gha                                # Docker layer caching via GitHub
Actions cache backend
cache-to: type=gha,mode=max
```

9.2 Adding Deployment Stages

```
deploy-staging:
  needs: build
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: staging
  steps:
    - name: Deploy to staging
      run: ./deploy.sh staging ${github.sha}

integration-tests:
  needs: deploy-staging
  runs-on: ubuntu-latest
  steps:
    - run: ./run_e2e_tests.sh https://staging.example.com

deploy-production:
  needs: integration-tests
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: production      # requires manual approval per Section 7.2's
                                # protection rules – this is the Continuous Delivery gate
  steps:
    - name: Deploy to production
      run: ./deploy.sh production ${github.sha}
```

This full chain — lint/test/scan in parallel → build once → deploy to staging → integration test against the REAL staging deployment → gated production deploy — is the canonical shape most mature pipelines converge on, directly implementing the pipeline stages described conceptually in Section 2.3.

9.3 Monorepo-Aware Pipelines (Path Filtering)

```
on:
  push:
    branches: [main]

jobs:
  changes:
    runs-on: ubuntu-latest
    outputs:
      api: ${ steps.filter.outputs.api }
      frontend: ${ steps.filter.outputs.frontend }
    steps:
      - uses: dorny/paths-filter@v3
        id: filter
        with:
          filters: |
            api:
              - 'services/api/**'
            frontend:
              - 'services/frontend/**'

  build-api:
    needs: changes
    if: needs.changes.outputs.api == 'true'
    runs-on: ubuntu-latest
    steps: [...]

  build-frontend:
    needs: changes
    if: needs.changes.outputs.frontend == 'true'
    runs-on: ubuntu-latest
    steps: [...]
```

This directly implements the “selective CI” capability referenced as a monorepo requirement back in Part 02, Section 13.2 — only rebuilding/testing the services actually affected by a given change, essential for keeping CI fast as a monorepo grows.

9.4 Tagging Artifacts with Git SHA for Traceability

```
- name: Build and tag image
  run: |
    docker build -t myapp:${ github.sha } .
    docker tag myapp:${ github.sha } myapp:${ github.ref_name }-latest
```

WHY THE GIT SHA, SPECIFICALLY

Tagging every built artifact with the exact Git commit SHA it was built from creates perfect, unambiguous traceability: given any running artifact in any environment, you can immediately answer “exactly which commit is this, and what changed since the last

deploy” — essential for incident response (Part 01, Section 16) and for implementing the “build once, promote the same artifact” principle from Section 2.4, since the SHA-tagged image is immutable and uniquely identifies its exact source.

10. Advanced Workflow Patterns

10.1 Job Outputs

```
jobs:
  determine-version:
    runs-on: ubuntu-latest
    outputs:
      version: ${ steps.get_version.outputs.version }
    steps:
      - id: get_version
        run: echo "version=$(cat VERSION)" >> "$GITHUB_OUTPUT"

  build:
    needs: determine-version
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building version ${ needs.determine-version.outputs.version }"
```

10.2 Dynamic Matrices

```
jobs:
  discover:
    runs-on: ubuntu-latest
    outputs:
      services: ${ steps.find.outputs.services }
    steps:
      - id: find
        run: |
          services=$(ls services/ | jq -R -s -c 'split("\n")[:-1]')
          echo "services=$services" >> "$GITHUB_OUTPUT"

  build:
    needs: discover
    strategy:
      matrix:
        service: ${ fromJson(needs.discover.outputs.services) }
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building ${ matrix.service }"
```

This pattern — discovering the matrix values dynamically at runtime rather than hardcoding them — scales a single workflow definition across however many services/packages actually exist in a repo, without editing the YAML every time a new one is added.

10.3 Conditional Logic Patterns

```
# Run only on tags matching a release pattern
if: startsWith(github.ref, 'refs/tags/v')

# Run only if the previous job succeeded AND this is not a fork PR
if: success() && github.event.pull_request.head.repo.full_name == github.repository

# Always run this step (e.g., a cleanup step), even if earlier steps failed
if: always()

# Run only if a previous step specifically failed (for failure notifications)
if: failure()
```

10.4 Composing It All: A Release Workflow

```
name: Release

on:
  push:
    tags: ["v*"]

permissions:
  contents: write
  id-token: write          # for OIDC-based artifact signing/publishing

jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build release artifacts
        run: ./build_release.sh ${github.ref_name}
      - name: Create GitHub Release
        uses: softprops/action-gh-release@v2
        with:
          files: dist/*
          generate_release_notes: true      # auto-generates notes from merged PRs
        since the last tag
```

This ties directly back to Part 02’s coverage of annotated tags and SemVer (Section 10) — pushing a properly formatted version tag is what triggers the entire release automation here, closing the loop from “tag a release” to “artifacts are built, signed, and published” with zero manual steps.

11. CI/CD Security

11.1 The CI/CD Pipeline as an Attack Surface

A CI/CD pipeline typically has broad access: it can read your source code, often has credentials to deploy to production, and executes code with relatively little scrutiny compared to the application code it's shipping. This makes it a high-value target — a compromised pipeline can be a direct path to compromising production, which is why CI/CD security deserves deliberate attention, not an afterthought. Part 26 covers this topic and broader supply-chain security in full depth; this section covers the GitHub-Actions-specific practices that belong here.

11.2 Least Privilege, Applied to Actions

```
permissions:
  contents: read          # set the MOST RESTRICTIVE default at the workflow level...

jobs:
  comment-on-pr:
    permissions:
      pull-requests: write    # ...and grant ADDITIONAL scopes only on the specific
job that needs them
    runs-on: ubuntu-latest
    steps: [...]
```

Set `permissions: {}` or a minimal `contents: read` at the workflow level as the default, and escalate only specific jobs that genuinely need more — never leave every job with the broad default permissions GitHub grants automatically unless a specific need justifies it.

11.3 The `pull_request_target` Trap, Explained in Full

```
# DANGEROUS PATTERN – do not do this
on:
  pull_request_target:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${ github.event.pull_request.head.sha } # checks out the FORK's
untrusted code...
      - run: npm ci && npm test                        # ...then EXECUTES it,
with full repo secrets available
```

Because `pull_request_target` runs with the base repo's elevated permissions and full secret access REGARDLESS of where the PR originates, checking out and then executing the PR's own (potentially malicious) code in this context hands an attacker your secrets and write access, simply by opening a PR from a fork containing a malicious `package.json` postinstall script or test file. If a workflow genuinely needs `pull_request_target` (e.g., to label/comment using elevated permissions), it must NEVER also check out and execute the PR head's code — keep any logic that touches untrusted fork content strictly within a separate `pull_request`-triggered workflow with the normal restricted token instead.

11.4 Supply-Chain Basics for Pipelines

- **Pin actions to a commit SHA**, not a mutable tag (Section 6.1) — the single highest-leverage practice against a compromised/hijacked Marketplace action.
- **Review third-party actions' source** before adopting them for anything security-sensitive — an action is just code that runs with your workflow's permissions and secret access.
- **Enable Dependabot for GitHub Actions** (`.github/dependabot.yml` with `package-ecosystem: "github-actions"`) so pinned SHAs get automated update PRs, keeping the security benefit of pinning without the maintenance burden of manually tracking upstream releases.
- **Scan dependencies and container images** for known vulnerabilities as a CI gate (Part 26 covers SAST/SCA/container scanning tooling in depth).

11.5 Branch Protection as a CI/CD Security Control

Tying back directly to Part 02, Section 14.4: requiring CI status checks to pass (and branches to be up to date) before merge is itself a security control, not just a quality one — it ensures security scanning steps (secret scanning, dependency audits, SAST) in your pipeline cannot be silently bypassed by merging around a failing check.

12. Monitoring and Observability of Pipelines

12.1 Built-In Visibility

- The **Actions tab** shows every workflow run, its status, duration, and full logs per step.
- **Checks API** integration surfaces pass/fail status directly on commits and PRs — the same mechanism branch protection (Part 02) depends on.
- **Workflow run badges**
(`![CI](https://github.com/org/repo/actions/workflows/ci.yml/badge.svg)`) give an at-a-glance status indicator, commonly embedded in a README.

12.2 Notifications

```
- name: Notify Slack on failure
  if: failure()
  uses: slackapi/slack-github-action@v1
  with:
    payload: |
      {"text": "CI failed on ${github.ref_name}: $
      {{ github.event.head_commit.message }} - ${github.server_url}}/$
      {{ github.repository }}/actions/runs/${github.run_id}"}
  env:
    SLACK_WEBHOOK_URL: ${secrets.SLACK_WEBHOOK_URL}
```

PRODUCTION PRACTICE: ALERT ON THE RIGHT SIGNAL

Notifying on EVERY CI run (pass or fail) trains a team to ignore notifications entirely within a week. Alert specifically on: failures on protected branches (`main` , release branches — not every feature branch), and especially on DEPLOYMENT failures, which need immediate attention. Reserve PR-level CI failure visibility for the PR itself (already visible via the Checks API) rather than adding noisy chat notifications for routine, author-visible failures.

12.3 Pipeline Metrics Worth Tracking

Metric	Why it matters
Lead time for changes	Time from commit to production deploy — a core DORA metric, reflects overall pipeline + process efficiency
Deployment frequency	How often you successfully ship to production — another core DORA metric
Change failure rate	Percentage of deployments causing a production incident/rollback
Time to restore service	How fast you recover from a failed deployment — tightly coupled to rollback/rollforward tooling quality
CI pipeline duration	Directly impacts developer feedback loop speed and iteration velocity
Flaky test rate	Tests that fail intermittently without a real code change erode trust in CI and should be tracked and fixed, not just re-run until green

These four DORA (DevOps Research and Assessment) metrics are widely used as the standard, industry-recognized way to measure software delivery performance — worth knowing by name, as they come up frequently in both tooling (many platforms report them natively) and in engineering-leadership conversations about pipeline health.

13. Troubleshooting GitHub Actions

13.1 A Systematic Approach

```
gh run list --workflow=ci.yml --limit 5      # recent runs for a specific workflow
gh run view <run-id>                        # summary: which jobs/steps passed
or failed
gh run view <run-id> --log-failed
# pull ONLY the failed step's logs directly into the terminal – fastest path to the
actual error
```

13.2 Common Failure Scenarios

Scenario: “Works locally, fails in CI”

Diagnosis checklist: Environment differences are the overwhelming majority cause — compare: exact dependency versions (did you run `npm ci / pip install -r requirements.txt` with a committed lockfile, or did CI resolve different versions than your local cache?), environment variables/secrets that exist locally but aren’t configured in CI, OS differences (if testing on `ubuntu-latest` but developing on macOS — path separators, case sensitivity on the filesystem), and timezone/locale differences affecting date-dependent tests.

Scenario: A step can’t find a file that “should” be there

```
steps:
  - run: cat README.md      # fails: "No such file or directory"
```

Diagnosis: The most common cause is a MISSING `actions/checkout@v4` step — unlike many CI systems, GitHub Actions does NOT automatically check out your repository; without an explicit checkout step, the runner’s working directory starts essentially empty.

Scenario: A secret isn’t available in a step

Diagnosis checklist: Is this a `pull_request` event from a FORK? (Secrets are deliberately withheld from fork PRs for security — Section 7.1/4.2.) Is the secret scoped to a specific ENVIRONMENT that this job doesn’t declare via `environment:` ? Is the secret name mistyped (fails silently as an empty string, not a hard error) — always double check exact spelling against the Settings page.

Scenario: A job that should run in parallel seems to wait

Diagnosis: Check for an implicit `needs:` dependency, or runner capacity limits — self-hosted runner pools or GitHub-hosted concurrent job limits (which vary by plan) can queue jobs even with no explicit `needs:` relationship, simply because no runner is free yet. `gh run view <run-id>` shows queued vs running state per job clearly.

Scenario: Matrix build partially fails and you can't tell which combination

```
gh run view <run-id> # lists each matrix job individually, e.g.,  
"test (ubuntu-latest, 20)" – click/view the SPECIFIC failing combination, not just the  
parent job
```

13.3 Debug Logging

```
# Repository secret: ACTIONS_STEP_DEBUG = true  
# Enables extremely verbose step-level debug output across ALL workflows in the repo
```

```
gh workflow run ci.yml -f debug=true # if your workflow_dispatch defines a debug  
input, as shown in Section 4.1
```

14. Best Practices and Anti-Patterns

GOOD PRACTICES

- **Fail fast:** order jobs/steps from cheapest-and-fastest to most-expensive-and-slowest (Section 2.3).
- **Pin action versions to a SHA** (Section 6.1/11.4) — security AND reproducibility.
- **Use OIDC instead of long-lived cloud credentials** wherever the cloud provider supports it (Section 7.4) — this should be the default for any new pipeline today.
- **Build once, promote the same artifact** through every environment (Section 2.4/9.4) — never rebuild per environment.
- **Keep CI fast** — a slow pipeline (15+ minutes for routine PR feedback) actively degrades team velocity and encourages risky workarounds (skipping checks, batching unrelated changes). Parallelize aggressively, cache dependencies, and use path filtering in monorepos.
- **Centralize shared pipeline logic** via reusable workflows (Section 6.3) once you have more than a couple of similar repos.
- **Make flaky tests a tracked, fixed problem**, not something silently re-run until green.

ANTI-PATTERNS

- **Long-lived cloud credentials stored as repo secrets** when OIDC federation is available — a needless, avoidable standing risk.
- **`pull_request_target` combined with checking out and running fork PR code** (Section 11.3) — a well-documented, serious vulnerability pattern.
- **Mutable tag pinning for third-party actions** (`@v4` instead of a SHA) on anything security-sensitive.
- **Workflows that always exit 0** regardless of actual test/build results — defeats the entire purpose of CI, exactly the anti-pattern called out for Bash scripts in Part 01, Section 13.8, now at the pipeline level.
- **No `concurrency: control` on PR workflows** — wastes runner minutes re-running superseded commits.
- **One giant monolithic workflow file with no job parallelism** — serializes work that could run concurrently, needlessly slowing every single run.
- **Treating a rebuilt-per-environment artifact as equivalent** to the one that was actually tested (Section 2.4) — undermines the entire guarantee CI/CD is supposed to provide.

15. Hands-On Labs

LAB ENVIRONMENT

All labs need a free GitHub account and a test repository. Labs 4-5 benefit from a free-tier AWS account for the OIDC lab, but can be adapted conceptually without one if unavailable.

Lab 1 — Your First Multi-Job Workflow

1. Create a new repo with a trivial app (even a single script with a test file is enough).
2. Write a workflow with three jobs: `lint`, `test`, and `build`, where `build` uses `needs:` to wait for both others.
3. Confirm in the Actions tab that `lint` and `test` run in PARALLEL, and `build` only starts after both succeed.
4. Deliberately break the test so it fails, and confirm `build` is correctly skipped as a result.

Lab 2 — Matrix Builds and Path Filtering

1. Extend Lab 1's `test` job into a matrix across at least 2 OS values and 2 version values of your runtime, and confirm it expands into the correct number of parallel jobs in the Actions UI.
2. Set `fail-fast: false` and deliberately break ONE matrix combination only — confirm the others still complete and report independently.
3. Add a second, unrelated directory to the repo (e.g., `docs/`) and configure `paths-ignore` so pushes touching only that directory don't trigger the workflow at all — verify with a test commit.

Lab 3 — Reusable Workflow

1. Create a reusable workflow (`workflow_call` trigger) that accepts an `environment` string input and a secret, and simply echoes them (simulating a deploy step).
2. Call it from a second workflow in the SAME repo with two different jobs, passing `staging` and `production` respectively.
3. Convert one of the composite “setup” steps from Lab 1 into a proper composite action under `.github/actions/`, and use it from both workflows.

Lab 4 — Secrets, Environments, and Approval Gates

1. Create two environments in repo settings: `staging` (no protection) and `production` (require a reviewer — yourself).

2. Add an environment-scoped secret to each with a different fake value (e.g., `DEPLOY_TARGET=staging-server` / `prod-server`).
3. Write a workflow with two sequential deploy jobs targeting each environment, and confirm the `production` job pauses for manual approval in the Actions UI, while `staging` does not.

Lab 5 — OIDC to AWS (or conceptual walkthrough if no AWS account)

1. In an AWS account, create an IAM OIDC identity provider for `token.actions.githubusercontent.com` and an IAM role with a trust policy scoped to your specific repo and branch (Section 7.4).
2. Grant the role a minimal policy (e.g., read-only access to one S3 bucket).
3. Write a workflow using `aws-actions/configure-aws-credentials` with `role-to-assume`, and confirm it can successfully call `aws s3 ls` with NO access keys stored anywhere in the repo.
4. Inspect the actual OIDC JWT claims (the role's CloudTrail event after assumption shows the federated subject) and confirm they match exactly the repo/branch you expected — understand what would happen if you tried running the same workflow from a different branch.

Lab 6 — Troubleshooting Drill

1. Deliberately introduce three separate common failures into a working pipeline, one at a time: (a) remove the `actions/checkout` step, (b) reference a secret that doesn't exist, (c) create an accidental circular/missing `needs:` dependency.
 2. For each, use `gh run view --log-failed` to diagnose the exact cause before fixing it, and write one sentence per scenario describing the error message pattern that identifies each root cause.
-

16. Command & Reference Cheat Sheet

16.1 gh CLI for Actions

Command	Purpose
<code>gh run list --workflow=ci.yml</code>	Recent runs of a specific workflow
<code>gh run view <id></code>	Run summary, job/step status
<code>gh run view <id> --log-failed</code>	Pull only the failed step's logs
<code>gh run watch</code>	Live-follow the current run
<code>gh workflow run <name> -f key=value</code>	Manually trigger with inputs
<code>gh workflow list</code>	All workflows in the repo

16.2 Common Trigger Patterns

Trigger	YAML
On push to main	<code>on: {push: {branches: [main]}}</code>
On any PR to main	<code>on: {pull_request: {branches: [main]}}</code>
On a version tag	<code>on: {push: {tags: ["v*"]}}</code>
Nightly	<code>on: {schedule: [{cron: "0 2 * * *"}]}</code>
Manual, with input	<code>on: {workflow_dispatch: {inputs: {...}}}</code>
Callable by other workflows	<code>on: {workflow_call: {inputs: {...}, secrets: {...}}}</code>

16.3 Key Contexts

Expression	Value
<code>\${{ github.sha }}</code>	Commit SHA that triggered the run
<code>\${{ github.ref_name }}</code>	Branch or tag name
<code>\${{ github.actor }}</code>	Who triggered the run
<code>\${{ secrets.NAME }}</code>	A configured secret
<code>\${{ needs.job_id.outputs.X }}</code>	Output from a dependency job
<code>\${{ matrix.X }}</code>	Current matrix combination value

16.4 Security Quick Reference

Practice	Why
Pin actions to a commit SHA	Immune to a moved/compromised tag
<code>permissions:</code> set explicitly, minimal	Least privilege for <code>GITHUB_TOKEN</code>
OIDC instead of long-lived cloud keys	No standing credential to leak
Never combine <code>pull_request_target</code> + checkout of PR head + execution	Prevents secret exfiltration via a malicious fork PR
Self-hosted runners never on public repos without isolation	Prevents arbitrary code execution via a public PR

17. Production Readiness Checklist

- ☐ Every workflow sets explicit, minimal `permissions:` rather than relying on broad defaults
- ☐ All third-party Marketplace actions are pinned to a commit SHA, not a mutable tag
- ☐ Cloud deployments use OIDC federation, not long-lived access keys stored as secrets
- ☐ `production` (and any sensitive) environment has required-reviewer protection configured
- ☐ No workflow combines `pull_request_target` with checking out and executing the PR head's code
- ☐ CI pipeline stages are ordered fast-to-slow (lint/fast checks before slow integration tests)
- ☐ `concurrency` is configured on PR-validation workflows to cancel superseded runs
- ☐ Built artifacts are tagged with an immutable identifier (Git SHA) and promoted as-is through environments, never rebuilt per environment
- ☐ Required status checks are enforced via branch protection (Part 02), so CI/security gates cannot be bypassed by merging around them
- ☐ Dependabot (or equivalent) is enabled for both application dependencies and GitHub Actions versions
- ☐ Failure notifications are scoped to meaningful events (protected-branch/deploy failures), not every routine PR failure
- ☐ Flaky tests are tracked and fixed, not routinely re-run until green
- ☐ A documented decision exists for Continuous Delivery vs Continuous Deployment per environment, matching the team's actual test-coverage and risk tolerance